

Making Post Quantum Key Exchange Efficient: An Implementation with the MLS Protocol

Noah Greene
noah.greene@nps.edu

Britta Hale
britta.hale@nps.edu

Abstract

The development of quantum computing technology poses a serious and credible threat to modern public-key cryptography, which is a pillar of secure communications. Post quantum cryptographic (PQC) algorithms can protect against this threat, but key establishment protocols supporting PQC algorithms pose non-trivial overhead costs. Prior proposals have pointed to the use of strategic traditional/PQC protocol combinations as a means of balancing performance and security, namely as an amortization of PQC overhead. This work provides the first benchmarking of this method within the context of the Messaging Layer Security (MLS) protocol. Comparative metrics include group size, CPU cycles, bytes, and runtime. The results show substantial overhead savings across the board when compared to a simple post-quantum cipher suite use, and marginal increase over traditional cipher suite performance when amortized. At small group sizes such as 1-to-1 channels, the method performs comparably to MLS using a traditional cipher suite. This work shows viability of deploying PQC solutions for constrained settings and achieving PQC security with efficiency.

I. Introduction

The growing threat of Cryptographically-Relevant Quantum Computers (CRQC) against traditional asymmetric cryptography creates an urgent need for the widespread adoption of post-quantum cryptographic (PQC) solutions. Such CRQCs are capable of breaking modern public-key algorithms, leading to wide-spread effects on cryptographic security, including putting at risk the security of data that is currently protected by classical encryption. To address this threat, new asymmetric algorithms have been finalized by NIST including ML-KEM (FIPS 203 [1]) as a post-quantum KEM, and ML-DSA (FIPS 204 [2]) as a post-quantum digital signature algorithm. These standards are being incorporation into key exchange protocols, such as TLS, to provide network security. However, algorithm overhead, especially in the form

of impacts on bandwidth, raise a question on how to achieve efficient PQC key exchange. Prior work has proposed an approach to this problem, offering amortization of the PQC overhead [3]. We will term this approach to the protocol security Amortized PQ (APQ). This work provides the first public testing results for the APQ method in the context of the Messaging Layer Security (MLS) protocol. These results are not optimized, but presented as an early comparison point to other PQC methods.

The MLS protocol is a group messaging protocol that seeks to provide strong security properties with Forward Secrecy, Post-Compromise Security, as well as efficient scalability for group communications. It was standardized by the Internet Engineering Task Force (IETF) in 2023. MLS functions using an asynchronous key update ratchet, i.e., a continuous key agreement (CKA), and is based on a Key Encapsulation Mechanism (KEM) which provides smooth integration of the FIPS PQC KEM.

Within an MLS group, participants contribute to a shared secret and are able to verify identities of other group members. When secrets need to be refreshed, such as when group membership changes, the key updates drive all legitimate members to a new shared secret, which is considered a new group state, referred to as an *epoch*. Users only have access to the internal secrets of epochs which they are members of, which keeps new members from accessing previous epochs, as well as keeping former members from accessing new epochs.

As with other protocols, MLS implementations are now transitioning to use of PQC algorithms and dealing with the FIPS PQC algorithms' well widely recognized size and overhead increase in comparison to traditional modern standards, the need for PQC migration has motivated the development of a type of hybrid amortization approach (i.e., APQ) that aims to provide a middle ground between post-quantum security and the efficiency of traditional cryptography. It was first proposed in the context of the MLS protocol (APQ-MLS) [3].

The asynchronous design of MLS enables further potential

for efficiencies within the context of PQC. APQ-MLS aims to offer low-overhead PQC security by linking two parallel MLS sessions, one classic and the other PQC, with different key evolution frequencies, thus *spreading* out the associated overhead.

With a PQC-capable MLS, APQC-MLS can provide CKA capabilities to meet the quantum threat. This is done by cryptographically linking the secret shared by a traditional session of the MLS protocol with the secret shared by a PQC session of MLS. This linking can be achieved by using the MLS export function to generate a PQC-secure secret which the traditional session can import as a pre-shared key (PSK). Using a PQ-secure value as a PSK incorporates the PQC security into the traditional session, allowing it to carry the bulk of the group’s message traffic with the efficiency of traditional encryption. This efficiency-security trade off can be modulated with the frequency with which the PQC secret is updated, allowing applications with more stringent PQC requirements to incur a higher computational cost to achieve stronger PQC security which in theory still beats out the computational cost of using a PQC MLS session exclusively. This work successfully showed the practical viability of efficient PQC secure communications through the APQC-MLS protocol.

While there the theoretical advantages of amortizing overhead are clear, APQ-MLS protocol lacks any formal implementation and performance data to confirm the security-performance trade-off it seeks to achieve, both of which are barriers to its real-world use as a standard.

This work aims to provide a concrete analysis of the performance of the APQC-MLS Combiner protocol measured against MLS running traditional and PQC cipher suites. After intensive development to integrate a PQ Key Encapsulation Mechanism into OpenMLS, the Rust implementation of MLS, it was possible to implement an approximation of APQC-MLS in Rust. The code was used to simulate simplified long-term runs of the protocol with different PQC **Update** frequencies, as well as to simulate the same length runs of MLS with traditional and PQC cipher suites.

II. Background and Related Work

In the realm of secure communications, session-based protocols are a common approach. Transportation Layer Security (TLS) [4] is a widely used communications protocol which performs AKE during its handshake subprotocol using Ephemeral Diffie-Hellman (DH) key exchange. There is currently a draft RFC integrating Hybrid Public Key Encryption (HPKE) into TLS [5]. IPsec [6] similarly uses DH variants for key exchange. Such protocols are being adapted to include PQC capability. Efforts are ongoing to integrate Key Encapsulation Mechanisms (KEMs) into TLS [7] which could allow for use of ML-KEM [8] or a hybrid KEM [9], [10]. Similar efforts are ongoing for IKEv2

[11], [12], SSH [13]. When considering the PQC variant² of handshakes for current standards such as TLS and SSH, overhead can increase up to 50% for SSH and up to 300% for TLS, depending on the PQC algorithms used [14].

All of the above protocols are session-based, which means that after the initial AKE, often referred to as a handshake, the key material is not refreshed without re-doing the handshake, which starts a new session.¹ The security impact of the session-based approach is that a session cannot recover from short-term key compromise, though a new session can be re-established if long term keys remain secure [15]. The capability to recover from a short-term key compromise without the need to re-establish a new session is called post-compromise security (PCS) [16], and it is a desirable quality and is often present in long-term, continuously evolving session protocols where a session might last years. Such protocols are termed continuous key agreement (CKA) protocols.

CKA protocols operate through periodic refresh of keying material within the context of the session, breaking the entire session into segments called epochs that represent individual cryptographic states. CKA protocols are also known as ratcheted key exchange protocols [15]. Well designed CKA protocols offer PCS as well as an additional property called forward secrecy (FS), which is the quality of past messages being secure in the event of a compromise [17]. CKA is the cornerstone of modern group messaging protocols, with ongoing improvements and research since its inception [15], [18]–[25]. Some CKAs are also making advances in PQC capabilities [26], [27]. One existing CKA protocol relevant to this work is the Messaging Layer Security (MLS) protocol [28], which will be discussed in depth in Section III.

III. Preliminaries, MLS, and Post-Quantum MLS

MLS [28] is a communications protocol designed to facilitate group CKA to achieve secure scalable group messaging. It performs CKA for group members during a session by generating a shared secret with input from all members and uses that to seed its key schedule. MLS is organized according to groups and epochs, with the group being a “logical collection of clients that share a common secret value at any given time” [28], and epochs being the time segments during which the group’s CKA remains on a particular shared secret. The epoch is advanced periodically according to application security requirements, or when group membership changes.

MLS builds on the CKA concepts established in the Signal protocol for ratcheting to support more two parties. This is achieved through a tree-based Key Encapsulation Mechanism

¹Exceptions apply to session resumption options, which are a more restricted use case.

(KEM) structure [29]. Group members are assigned to leaf nodes within the tree structure, each of which holds a public key and a view of the keys that lead to the root. Updates to the tree thus require only $\log(N)$ update messages, rather than N messages [28], and the synchronicity is unnecessary because members can independently arrive at the shared secret at the root of the tree without requiring input from any other members.

The addition of the commit secret prevents former members or attackers who hold the previous epoch secret from learning the current secret, and newly added members lack the previous epoch secret, which ensures post-compromise security and forward secrecy respectively. The epoch secret is then used to generate a tree of secret values corresponding to the ratchet tree, such that each member has a secret for use with the current group and epoch.

The ratchet tree structure used by MLS makes scalability far more efficient, as group size scales the computational efforts of deriving and updating shared keys scale logarithmically instead of the linear scaling that other prominent strategies such as Signal face [28].

A. PQC in MLS

MLS uses several cryptographic mechanisms to achieve its security goals, with the most relevant to this work being Hybrid Public Key Encryption (HPKE). HPKE is a scheme that combines asymmetric and symmetric key cryptography to securely generate and exchange symmetric keys, utilizing an asymmetric Key Encapsulation Mechanism (KEM), an Authenticated Encryption with Additional Data (AEAD) algorithm, and a Key Derivation Function (KDF).

1) *KEM in HPKE*: The KEM is the core algorithm for HPKE. The following provides the KEM Definition 1 and the associated correctness Definition 2.

Definition 1 (Key Encapsulation Mechanism (KEM) [30]).

A KEM is a triple

$(KGen, Encap, Decap)$:

- $KGen(1^\kappa) \rightarrow (pk, sk)$: On input security parameter κ **return** key pair pk, sk (probabilistic algorithm),
- $Encap(pk) \rightarrow (C, K)$: On input pk **return** ciphertext $C \in \mathcal{C}$ and key $K \in \mathbf{K}$ (probabilistic algorithm).
- $Decap(sk, C) \rightarrow K/\perp$: On input sk and ciphertext C **return** key $K \in \mathbf{K}$ or rejection $\perp \notin \mathbf{K}$ (deterministic algorithm).

Definition 2 (KEM Correctness [30]). We say a KEM is $(1 - \delta)$ correct, if for every key pair $(pk, sk) \leftarrow \$KGen(1^\kappa)$ and every encapsulation $(C, K) \leftarrow \$Encap(pk)$ we have:

$$\Pr[K' \neq K | K' \leftarrow Decap(sk, C)] \leq \delta.$$

A KEM is perfectly correct if $\delta = 0$.

The KEM is particularly relevant to this work, as adaptation of the HPKE to use a PQC KEM is an intrinsic

component. KEMs consist of three component functions: a key generation function, an encapsulation function, and a decapsulation function. These functions are used to send a symmetric key between communication partners using asymmetric cryptography. The generation function creates a public/private key pair for the recipient, and the encapsulation provides a shared secret key to the sender and a copy of the shared secret within an encrypted ciphertext that can be safely transmitted to the receiver. The receiver then uses their private key to decapsulate the ciphertext and obtain the same shared secret as the sender.

KEMs make use of public-key cryptography and HPKE in particular currently makes use of Diffie-Hellman-based KEMs (DHKEM) [31], all of which are elliptic-curve variants. The result of this dependency is that HPKE and any protocols utilizing it are not quantum-safe currently, but could in theory be made quantum safe through use of a PQC KEM.

PQC KEMs are a growing subject of research, with the ML-KEM standard at the forefront amid other PQC and hybrid algorithms [32], [33], [34]. A PQC cipher suite for MLS must consist of an HPKE component that uses a PQC KEM, and optionally can include a digital signature component that uses a PQC digital signature. Neither MLS nor OpenMLS currently support any form of PQC cipher suite, which necessitated the implementation of at least one new cipher suite.

B. PQC MLS

Much like the efforts for other CKA protocols, research is moving to add PQC security to MLS. In order to achieve this, a PQC KEM must be used by the HPKE implementation to gain confidentiality during key exchange.

1) *Simple PQC MLS*: HPKE is specified to accept new KEM constructions given that they satisfy its security requirements [31], which means utilizing a PQC KEM within HPKE is dependent on implementation and requires no changes to the HPKE protocol itself to lend it PQC security. However, an addition is required specifically to allow Kyber, the predecessor to ML-KEM that was used for this work, to interface fully with HPKE. Kyber lacks a *DeriveKeyPair* functionality unlike the DH-based KEMs specified by HPKE [31], which prevents it from accepting an Input Keying Material (IKM) value to generate its keys as in DHKEM, making the drop-in replacement of Kyber for DHKEM not entirely straightforward. Thus an additional step is used to take the IKM value as a random number generator (RNG) seed and passing that to Kyber's *KeyGen* function, which successfully incorporates the IKM into Kyber and allows it to meet the specifications of the HPKE *DeriveKeyPair* function. While this does not represent a modification to HPKE as a protocol nor to Kyber as an algorithm, it was a necessary addition to allow the two to be integrated together. The simplest configuration is thus to incorporate a PQC

KEM into HPKE and use it with a new MLS cipher suite. This PQC KEM could be either hybrid, such as X-Wing [33], [35], or pure, such as ML-KEM [1] depending on the security and performance needs of the application.

The issue with simple PQC solutions is that the PQC KEM is in use for all key establishment and update operations. This means that while PQC security is present, the high overhead cost of PQC algorithms is incurred with every operation [3] – this applies to both use of ML-KEM and hybrid cipher suites like X-Wing. Thus the APQC-MLS Combiner seeks to strike a balance between performance and PQC security by combining a PQC MLS session with a traditional session in a way that can spread out the overhead of PQC key updates.

C. APQC-MLS Combiner

The APQC-MLS Combiner [3] is a draft RFC that defines a protocol for achieving post-quantum security while lessening the performance costs incurred by PQC KEMs. It proposes to achieve this through a protocol-level combination of traditional and post-quantum cryptography.

The APQC-MLS design dictates that for a given group of users who seek to form a group, two MLS sessions must be run in parallel. The group construction of the two sessions must have identical membership, while one session utilizes a traditional cipher suite and the other uses a post-quantum cipher suite. The APQC-MLS Combiner draft notes that there are two degrees of PQC security that can be achieved with the protocol, depending on the mode of operation. The mode relevant to this work is the Post-Quantum/Traditional (PQ/T) Confidentiality Only mode which only requires a PQC KEM for the PQC MLS session [3] and aims to provide protections against harvest-now-decrypt-later attacks, but not quantum impersonation. The PQ/T Confidentiality Only mode uses the PQC KEM to provide confidentiality against a Cryptographically-Relevant Quantum Computer (CRQC) by securing the all asymmetric key exchange between clients in a group. It also provides, as the APQC-MLS draft RFC states, PQC authenticity against group outsiders due to the inclusion of PQC keying material in the key schedule and use of AEAD ciphers [3].

The other mode, PQ/T Confidentiality + Authenticity mode, adds PQC authenticity to the Confidentiality Only mode via use of PQC signatures on key updates in the PQC session. This adds overhead to the operation of the PQC session but blocks the potential for a quantum impersonation attack on the key updates [3].

The APQC-MLS Combiner creates a cryptographic tie between the PQC group and the traditional group in order to take advantage of the traditional cipher suite’s performance for the majority of the group’s messages and **Updates** and using the PQC group to export a PQ-generated secret, which is integrated into the traditional group as a pre-shared key (PSK) **Proposal**. This injection of PQC security

gives the traditional session PQC protection that persists across **Updates** within the traditional session, and can be applied as often as desired by the application utilizing it. The only requirements for frequency in applying a PQC **Update** through use of such an exporter key into the traditional session are for changes in membership. This is because membership changes require the formation of a new cryptographic state for the group to keep past epochs confidential to members of those epochs only. By only performing the PQC computations for changes of group membership and an application-set frequency relative to the desired security level, the group’s operations are PQC-secured without the performance cost of regular PQC key updates. Note that increasing PQC **Update** frequency does not necessarily imply a proportional increase in PQC security. PQC keying material lifetimes are a deciding factor in the PQC security level achieved in both sessions and more frequent PQC key refresh does not always introduce practical security changes.

The APQC-MLS Combiner uses specific terminology to describe the two different forms of “**Updates**” used in its operation. A **Full Update** is an **Update** to the PQC group, the export of a new PQC secret, and the secret entering the traditional group’s key schedule as a PSK. The **Commit** itself, which is a protocol message sent to all members of the group, informs the member of the change and prompts them to proceed into the next epoch, so a **Full Update** results in both groups advancing to a new epoch. A **Partial Commit** occurs when the traditional group advances to a new epoch without the PQC group doing so, which is only allowed when there are no changes in membership.

The APQC-MLS Combiner is a flexible protocol. The degree of PQC security and performance cost are variably determined by the frequency of **Full Updates** performed during the session, which gives a great degree of control to the application in terms of the exact trade-off that is achieved. Three frequencies are measured in this work to establish a baseline understanding, but applications are by no means bound to those parameters in their own implementations.

IV. Methodology

The first phase of this work is comprised of software development to modify existing Rust libraries to support a PQC cipher suite to be tested. The second phase consists of the design and execution of the simulations for data gathering and analysis.

OpenMLS [36] is an open-source Rust implementation of MLS. A fork of the OpenMLS project served as the basis for the software development that supported this work. Presently OpenMLS does not support the entire set of cipher suites defined by the MLS specification, notably lacking any 256-bit security level options, although this is recommended in practice. The CPU and RAM specifications for the machine

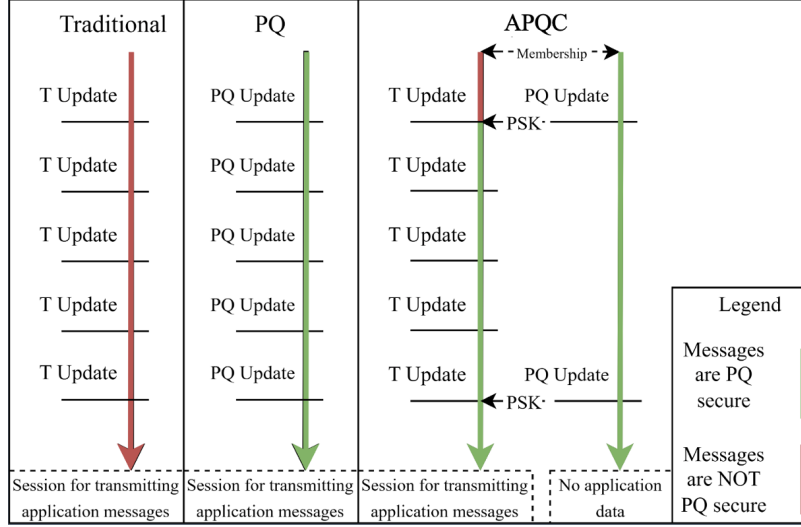


Fig. 1: Illustration of functional comparison of MLS, MLS with a PQC cipher suite, and APQC-MLS, with PQC and Traditional protections shown

used in the simulations are presented in Table I. Peer-reviewed publications covering performance testing of MLS implementations are nearly nonexistent. One thesis work [37] conducted a comparison of three MLS implementations as well as an implementation of the Signal protocol. We use a similar benchmarking methodology in this work.

TABLE I: CPU and RAM Specifications used for testing

Component	Manufacturer	Model	Details
Processor	AMD	FX(tm)-6300 Six-Core Processor	3500 Mhz 3 Cores 6 Logical Processors
RAM	Corsair	Vengeance Pro	4x8 GB 1600MHz DDR3

A. Experiment 1

Experiment 1 seeks to present three key metrics which serve as the basis of comparison for the long-term performance of APQC-MLS against MLS using a PQC cipher suite. Those metrics are:

- Execution time
- CPU cycles consumed
- Control Message sizes

Time-based performance data (execution time and CPU cycles) was gathered using the Rust benchmarking library criterion.rs [38]. This library is a powerful tool that provides rigorous and statistically significant performance data. It accomplishes this by running each benchmarked function 100 times after a 3-second warm-up interval during which the function is executed several times to warm up the machine's caches so as to provide a consistent level of performance across the 100 samples. It also explicitly blocks

compiler optimizations to ensure that the entirety of the function is executed during each sample, using a black box function on all parameters to the function being measured. A complication from using the Rust implementation and criterion to measure the simulations was that every session was freshly initiated for the simulation. Since CKAs only perform initiation once, including for potentially multi-year sessions, session initiation is not a realistic measure for the steady-state operation of the protocol. Thus, in practice the computation needed by an individual client during normal operation is far smaller than the simulations represent. Execution time is used as a measurable representation of the total computational work required to simulate an entire group's operations.

Epochs: The first controlled variable was the number of epochs iterated for each group simulation, set at 500 epochs as the constant total epoch iterations for each simulation. This was chosen as being a large enough number to expect the results distinguish minor differences between the different configurations. Consequently, 100 samples of 500 epochs each were calculated for the time-based measurements. For the message size measurements, a single run of the simulation code was executed. Statistical analysis was unnecessary for message size capture, as the messages are consistently the same size for a given set of simulation parameters.

Message Types: The second controlled variable was the types of messages in use. The setup of the group includes a one-time **Add** operation which creates a *Welcome* message, and sustained operations only utilized the self **Update**, which replaces a single user's keys within the key management tree [28], and the pre-shared key **Proposal** message, which was necessary for each **Full Update** within the context of

the APQC-MLS Combiner simulations only.

Group Size: Capturing the performance trends across group sizes requires sufficient data points. Group size, as measured by total number of members, directly affects the number of times a particular message must be processed in the simulation. Any crossover point between group size and protocol configuration at which one is only better than the other for a certain range of group sizes is a notable consideration for performance optimization in practice. Patterning from [37], the tests started with group sizes of 2, 3, 4, and 5. To more quickly scale the data up to a higher magnitude, the following intervals were used for group size: 10, 15, 20, 25, 30, 35, 40, 45, 50, 60, 70, 80, 90, 100. These intervals serve to establish the trend of computational effort and message sizes required to operate a group within each protocol configuration. All tests were performed using these group size intervals.

Update Type and Frequency: Five different protocol configurations were implemented for comparison. These configurations and their **Update** frequencies are displayed in Table II. The first, labeled “Traditional” in this work, was an unmodified MLS session using one of the built-in cipher suites, giving it no post-quantum security and theoretically the least overhead of the five configurations. The next, labeled “PQ” in this work, was a modified MLS session using a modified cipher suite developed for this work, which integrates a PQC KEM. This configuration represents the most frequent post-quantum key update frequency of the five, and therefore the best PQC PCS, as it refreshes its PQC security with every epoch update. The last three configurations are implementations of the APQC-MLS Combiner in PQ/T Confidentiality Only mode. These configurations are named and referenced according to the ratio of **Partial Updates** to **Full Updates**, e.g. APQC-10 performs 10 **Partial Updates** per **Full Update**. The configurations show the flexibility of the APQC-MLS combiner in modulating the security-performance trade off, requiring only a change of function parameter to switch between them, and represent the expected middle ground of performance between the Traditional and PQC configurations.

The following cipher suites were utilized for testing:

- 1) MLS_128_DHKEMX25519_CHACHA20POLY1305_SHA256_Ed25519
A standard supported MLS cipher suite.
- 2) MLS_128_KYBER_AES256GCM_SHA256_Ed25519
A cipher suite with a post-quantum KEM, developed in support of this work.

1) *Simulation:* The sequence for each group simulation consisted of the following setup operations:

- Generate client list
- Assign credentials
- Instantiate group object
- Create **Add** message and process across all clients

TABLE II: Cipher suite and ratio of PQC **Updates** to traditional **Updates** for each protocol configuration across 500 epochs

Configuration Name	Cipher Suite(s)	Total PQC Updates	Traditional Updates per PQC Update
Traditional	(1)	-	-
APQC-100	(1,2)	5	100
APQC-50	(1,2)	10	50
APQC-10	(1,2)	50	10
PQC	(2)	500	0

The setup would ideally occur within a separate measurement from the sustained operation, but the setup and sustained operation were not possible to separate. Sustained operation consists of the following, repeated 499 times for a total of 500 key rotations across the simulation:

- Create **Update** message and process across all clients
- Create **Commit** message and process across all clients

In the case of the APQC-MLS Combiner simulations, the following is repeated a number of times equal to the number of full **Updates** for that configuration as given in Table II.

- The following, repeated a number of times equal to the number given in the configuration name, in the traditional MLS session:
 - ◊ Create **Update** message and process across all clients
 - ◊ Create **Commit** message and process across all clients
- In PQC session, create **Update** message and process across all clients
- In PQC session, create **Commit** message and process across all clients
- PQC session exports a secret
- Traditional session proposes it as a PSK, process across all clients
- Traditional session creates **Commit** message and processes across all clients

B. Experiment 2 – In-Session Cycles

The goal of Experiment 2 is to assess the CPU cycles per epoch on average for each configuration once running. Experiment 1 included calculations for session initiation; however, under a CKA the session is initiated once and run long-term (even spanning network disconnections), as opposed to session-based protocols like TLS where session re-initiation occurs on a regular basis. Thus, Experiment 2 seeks to capture the CPU cycle differences under normal use. It closely follows Experiment 1 in most respects, with the following changes.

A new benchmark was written to gather the data. To measure the average runtime and therefore the CPU cycles consumed on average per-epoch in session for of each protocol configuration, the new benchmark was designed

to allow for the session instantiation to occur outside of the measurement window. As with the previous runtime measurements, this benchmark used Criterion to measure the execution 100 times following a 3-second cache warm-up period. The benchmark was set to measure three functions:

- Execution of a single epoch of traditional MLS with the traditional cipher suite. The single epoch comprised of the group owner initiating an **Update**, serializing the message, committing to the **Update**, and each client processing the message.
- Execution of a single epoch of traditional MLS with the PQC cipher suite. The single epoch operations were identical to those described for the traditional cipher suite.
- Execution of a single **Full Update** in the context of an APQC-MLS session. This consisted of the PQC group performing an **Update** exactly as described in the two functions above, exporting a PSK to the traditional group, and the traditional group proposing and then committing to the **Update** containing the PSK **Proposal**.

The measurement of these three functions allows us to represent the average cycles required for a given epoch in-session, on average for 500-epoch simulations. The average value across the 100 Criterion iterations provides the expected time and thus CPU cycles required to execute Traditional and PQC **Updates** directly. For the APQC configurations this cannot be done directly because an epoch for APQC-MLS requires different operations depending on whether it is advanced by a **Full Update** or a **Partial Update** and therefore an average is required. An average APQC-MLS epoch's execution can be calculated as the average of the cycles for the **Full Updates** (PQC plus Traditional) and **Partial Updates** (Traditional only) that take place across the 500-epoch simulation. Note that the code to simulate a **Partial Update** is identical to the code that updates the Traditional configuration, so the data collected for that configuration can be used for the calculations of the APQC configurations. To calculate the estimate for APQC-100, an average over 5 **Full Updates** and 495 **Partial Updates** were calculated. For APQC-50, an average over 10 **Full Updates** and 490 **Partial Updates** were calculated, and for APQC-10 an average over 50 **Full Updates** and 450 **Partial Updates** were calculated.

C. Abstractions and Limitations

Several abstractions were taken in the design and implementation of the MLS group simulation with which the performance data was measured. The first was the group construction itself, which was all performed on a single machine using a single Rust source file. The computational work of instantiating each client and storing its group data, as well as the processing of each message, was also performed

within the same source file. The development was absent of any threading or parallel processing, much like the design of Lenz's tests [37]. This abstraction was taken to reduce code complexity to fit within the scope of feasibility for this work, so the understanding that the simulations handle all events sequentially for each client must be acknowledged when interpreting the results.

Network simulation was deemed out of scope for the work as well, so there was no simulation of the transportation of network traffic, thus the code only goes as far as the serialization of messages in preparation to be transmitted, treating them as though transmission time was instantaneous.

Finally, the APQC-MLS draft RFC [3] denotes a requirement for an extension to MLS that must be supported for correct operation, adding a specialized data structure to ensure the groups are synchronized. Implementing this extension into the simulation was not feasible for the scoping and timeline for this work, so while it is a necessary element for practical implementations of APQC-MLS it was not present in the simulations. The data structure incorporated into MLS by APQC-MLS is unlikely to noticeably impact computation time.

Possible confounding variables include the computational noise introduced by the background operations of a standard windows 10 machine. The machine was chosen to provide a realistic environment under which MLS might be employed, and as such there were consistently background applications in operation during the simulation runs, and it is impossible to consider the impact of those background applications to be consistent nor negligible.

An additional confounding factor to consider is the construction of the group simulation functions in Rust. The implementation structure for the fully traditional and fully post-quantum simulations is inherently different from that of APQC-MLS, as that protocol introduces the need for two concurrent groups with a cross-group interaction, none of which occurs in the MLS-only simulations. A more accurate comparison might be found by considering the computational time required only to iterate the group(s) through 500 epochs after setup is complete, but it was not possible to implement such a measurement using Criterion.

V. Results and Analysis

This section presents the results of the testing performed according to the methodology outlined in the previous section. The data collected on the different configurations of APQC-MLS suggests that there is a wide range of **Full Update** frequencies available that achieve a predictable trade-off for post-quantum security and performance.

Confounding Variables: Unexpectedly, the APQC-100 and APQC-50 simulations outperformed the Traditional simulation in some cases. Specifically, the APQC-100 configuration was faster than Traditional in all tests except for

group sizes 60 and 70. Additionally, APQC-50 was the fastest simulation average for group sizes 2, 3, 60, 70, and 80, and was faster than Traditional for group sizes 4 through 50. In theory there is no reason this should be the case, so it is likely due to uncontrolled confounding variables in the simulation environment. The unexpected better performance of the APQC-100 and APQC-50 configurations in comparison to the Traditional configuration is likely the result of confounding factors in the experiment design and should not be used to conclude that the APQC-MLS Combiner performs faster than a traditional MLS session, with the interpretation being instead focused on the significant differences and trend-lines. The anomalous data can be seen in Table ??.

Experiment 1 – Speed Comparison

In Table III, the test data shows a clear growth in computation time across all group sizes as the protocol configuration used incorporates more frequent **Full Updates**. Each data point shown represents the average runtime result of the configuration at different group sizes, taken from a collection of $N = [100]$ samples. Figure 2 illustrates the growth rates of the execution time required for each protocol configuration, with the PQC simulation showing the fastest overall growth rate as group size increases.

TABLE III: Test Results for average runtime in seconds (average taken from $N = 100$ samples).

Group Size	Trad.	APQC-100	APQC-50	APQC-10	PQ
2	1.37	1.16	1.15	1.56	1.58
3	2.40	2.05	1.99	2.64	3.09
4	3.21	2.65	2.77	3.47	4.19
5	4.32	3.53	3.76	5.05	6.25
10	9.92	8.13	9.01	11.89	14.96
15	15.96	13.09	14.17	17.21	23.37
20	24.94	20.38	22.47	27.13	38.63
25	32.76	27.58	29.19	35.19	50.74
30	41.61	35.23	36.05	45.93	64.35
35	60.18	48.73	50.74	65.69	92.43
40	70.96	53.98	58.79	72.89	108.72
45	81.47	63.26	66.72	83.58	123.22
50	90.10	72.58	80.38	94.00	139.14
60	113.52	116.17	101.14	119.65	178.61
70	166.86	173.45	153.64	176.16	277.68
80	210.33	205.54	180.37	216.18	310.89
90	202.16	187.47	206.27	247.45	361.87
100	230.50	226.05	247.13	308.95	433.32

When the APQC configurations are compared to the PQC configuration, a clear performance improvement is visible. Beyond 100 clients, as well as at less-frequent **Full Update** rates than APQC-100, we can be confident the APQC variants continue to perform better than traditional MLS running a post-quantum cipher suite. Using data on Table IV there is a consistent advantage over the PQ-only

variant of 33%-50% for APQC-100, 40%-45% for APQC-50,⁸ and 20%-35% for APQC-10.

TABLE IV: Percentage performance speedup in seconds of APQC-100, APQC-50, and APQC-10 in comparison to PQC in the 500-epoch simulations

Group Size	% H-100 to PQC	% H-50 to PQC	% H-10 to PQC
2	26.48%	27.24%	0.99%
3	33.54%	35.62%	14.56%
4	36.86%	33.84%	17.27%
5	43.58%	39.76%	19.23%
10	45.62%	39.78%	20.48%
15	43.96%	39.38%	26.34%
20	47.24%	41.84%	29.79%
25	45.64%	42.47%	30.65%
30	45.26%	43.98%	28.63%
35	47.28%	45.11%	28.92%
40	50.35%	45.93%	32.95%
45	48.67%	45.85%	32.17%
50	47.84%	42.23%	32.44%
60	34.96%	43.37%	33.01%
70	37.54%	44.67%	36.56%
80	33.89%	41.98%	30.47%
90	48.19%	43.00%	31.62%
100	47.83%	42.97%	28.70%

Experiment 1 – CPU Comparison

CPU cycle data was derived from the simulation runtime data from Criterion. Cycle data was calculated by multiplying the execution runtime values with the testing machine’s specification clock rate of 3500 GHz (specifications in Table I). The calculated data is presented in Table V. As this is a unit conversion of the data from Table III, all analysis of the simulation runtime data is also applicable to this section. Percentage performance speedups in Table IV apply here as percentage fewer cycles required by the APQC configurations compared to PQC in the 500-epoch simulations.

Experiment 1 – Data Comparison

Table VI and Figure 3 show the average bytes that are simulated as being transmitted by the group owner per epoch, including the *Welcome* message transmitted to add members to the group. Averages were calculated using the same 500-epoch model used in the other tests, and for the APQC configurations represents the amortized cost of any **Full Updates** performed during those simulations. This does not represent the total network traffic for any given user of the protocol, as the delivery system and application messages were not simulated in this work; rather it shows a comparison across security approaches for traditional, PQ, and APQCs. Data trends show that APQC-MLS achieves a data transmission size comparable to Traditional MLS and far smaller than the PQC configuration.

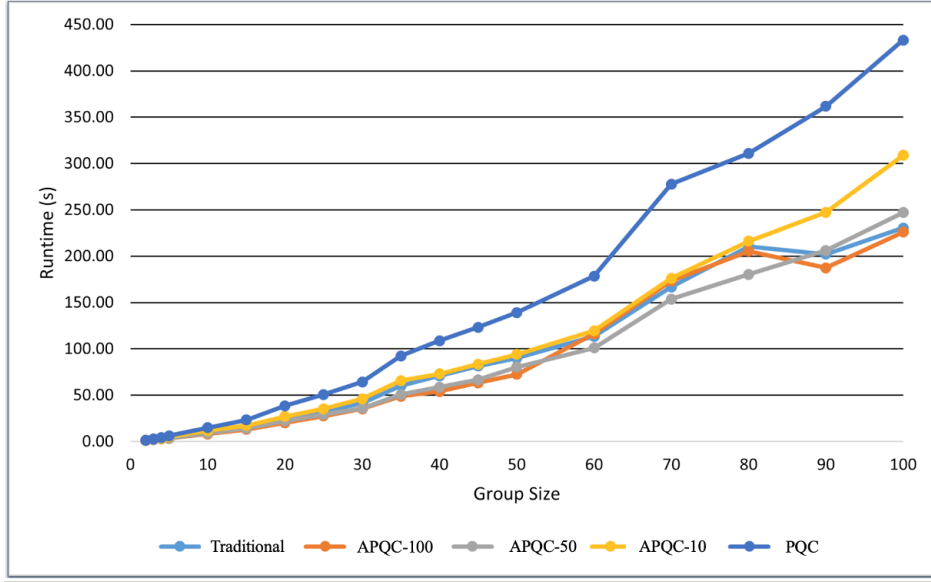


Fig. 2: Runtime in seconds vs. group size for 5 protocol configurations.

TABLE V: Average CPU cycles in billions consumed by protocol configuration based on 3500 MHz CPU spec and runtime data presented in Table III.

Group Size	Trad.	APQC-100	APQC-50	APQC-10	PQC
2	4.80	4.06	4.02	5.47	5.52
3	8.40	7.18	6.96	9.24	10.81
4	11.23	9.27	9.71	12.14	14.68
5	15.11	12.34	13.18	17.66	21.87
10	34.73	28.47	31.52	41.62	52.35
15	55.87	45.83	49.58	60.24	81.78
20	87.29	71.35	78.64	94.94	135.22
25	114.65	96.54	102.17	123.16	177.59
30	145.64	123.30	126.17	160.75	225.23
35	210.61	170.54	177.58	229.92	323.49
40	248.35	188.92	205.75	255.13	380.51
45	285.16	221.39	233.52	292.54	431.28
50	315.35	254.03	281.32	328.99	486.99
60	397.34	406.59	353.99	418.77	625.12
70	583.99	607.07	537.76	616.55	971.86
80	736.15	719.39	631.29	756.62	1,088.13
90	707.57	656.13	721.96	866.06	1,266.53
100	806.74	791.19	864.97	1,081.34	1,516.64

TABLE VI: Average bytes per epoch for MLS control messages sent by group owner, calculated across epochs for each protocol configuration.

Group Size	Trad.	APQC-100	APQC-50	APQC-10	PQC
2	515.80	557.74	593.28	877.60	2789.26
3	632.87	694.45	747.39	1170.91	4410.79
4	715.01	787.71	849.66	1345.26	5229.93
5	832.08	924.43	1003.78	1638.58	6851.47
10	1277.71	1434.17	1566.96	2629.28	11749.56
15	1688.41	1900.45	2078.29	3501.01	15845.26
20	2134.04	2410.23	2641.55	4492.11	20747.35
25	2544.74	2876.51	3152.88	5363.84	24843.05
30	2955.44	3342.79	3664.21	6235.57	28938.75
35	3401.07	3852.53	4227.39	7226.27	33836.84
40	3811.77	4318.81	4738.72	8098.00	37932.54
45	4222.47	4785.09	5250.05	8969.73	42028.24
50	4633.17	5251.37	5761.38	9841.46	46123.94
60	5454.57	6183.95	6784.08	11585.12	54317.33
70	6310.90	7159.97	7858.59	13447.55	63311.13
80	7132.30	8092.53	8881.25	15191.01	71502.53
90	7953.70	9025.11	9903.95	16934.67	79695.92
100	8775.10	9957.67	10926.61	18678.13	87887.32

Table VII illustrates the advantages of APQC-MLS over the PQC configuration in terms of bytes transmitted, with far less data being needed for operating APQC-MLS in comparison to the PQC configuration of MLS. The simulated configurations and group sizes show APQC-MLS data sizes between 68% and 88% smaller than the PQC configuration of MLS at the ratios selected (H-100, H-50, H-10).² This

makes a significant difference for applications with limited bandwidth where transmitting the larger messages required for PQC key rotation difficult.

A. Experiment 2 – In-Session Cycles

Table VIII shows the average CPU cycles required for executing an epoch across all five protocol configurations and the established group size testing range. The measurements were used to calculate the amortized cost for the periodic Full

²In practice, an application can select the most advantageous ratio for the security and functionality needs of the use case

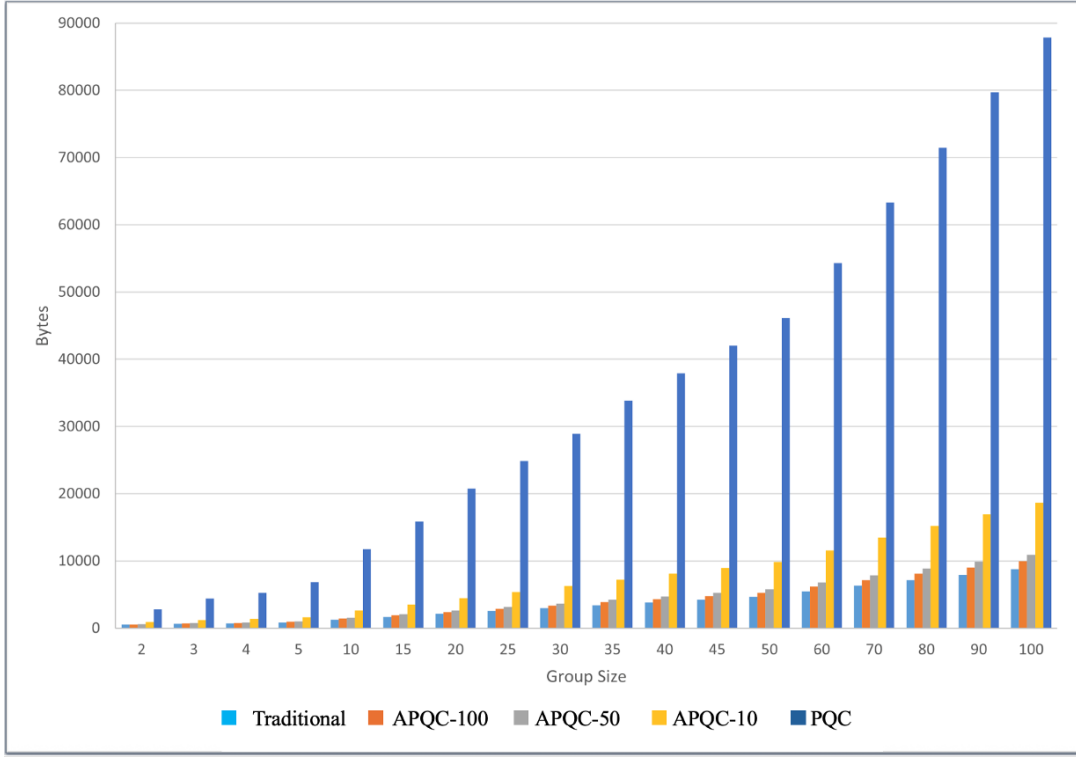


Fig. 3: Average per epoch bytes of control messages sent by group owner across 500 epochs for each protocol configuration.

TABLE VII: Percentage of data transmission improvement in bytes of APQC-100, APQC-50, and APQC-10 in comparison to PQC in the 500-epoch simulations.

Group Size	H-100 vs PQC	H-50 vs PQC	H-10 vs PQC
2	80.00%	78.73%	68.54%
3	84.26%	83.06%	73.45%
4	84.94%	83.75%	74.28%
5	86.51%	85.35%	76.08%
10	87.79%	86.66%	77.62%
15	88.01%	86.88%	77.91%
20	88.38%	87.27%	78.35%
25	88.42%	87.31%	78.41%
30	88.45%	87.34%	78.45%
35	88.61%	87.51%	78.64%
40	88.61%	87.51%	78.65%
45	88.61%	87.51%	78.66%
50	88.61%	87.51%	78.66%
60	88.62%	87.51%	78.67%
70	88.69%	87.59%	78.76%
80	88.68%	87.58%	78.75%
90	88.68%	87.57%	78.75%
100	88.67%	87.57%	78.75%

Update in the APQC configurations across 500 epochs. Note that the average cycles per epoch for the Traditional and PQC configurations is not theoretically influenced by the length of the session in the same way as in APQC configurations, so

a single 500-epoch sample was averaged per epoch for each configuration. These values were measured as execution runtime and converted to CPU cycles in the same way as for Table V in Experiment 1.

The in-session simulations show a clear computational advantage in terms of CPU cycles for the APQC-MLS configurations over traditional MLS with PQC a cipher suite. The data in Table VIII shows that an average epoch for the APQC-MLS Combiner is between 30-45% faster than the PQC configuration of MLS. This makes a significant difference for applications where computing power is limited in terms of speed or power consumption such as in embedded or cyber-physical systems.

These results indicate that APQC-MLS Combiner configurations provide performance advantages in terms of total computation time when compared to traditional MLS running with a PQ-KEM cipher suite. The three APQC variants provide post-quantum security to MLS with reduced overhead, even as the group size scales up. The security trade-off to achieve this is frequency of the PQC key rotation. However, interim traditional key updates to the key schedule do ensure PQC FS and traditional PCS at regular intervals, so the PQC PCS is the main delayed factor.

TABLE VIII: Average CPU cycles in millions consumed by protocol configuration based on 3500 MHz CPU spec and single-epoch runtime data. APQC configurations are calculated by averaging the data for **Full Updates** and **Partial** (Traditional-only) **Updates** in a ratio corresponding to that executed in each simulation.

Group Size	Trad.	APQC-100	APQC-50	APQC-10	PQC
2	5.14	5.23	5.32	6.02	6.94
3	9.55	9.71	9.86	11.10	13.86
4	12.47	12.68	12.89	14.55	18.52
5	18.81	19.11	19.41	21.78	27.77
10	41.86	42.60	43.35	49.32	69.00
15	63.57	64.71	65.84	74.92	114.98
20	100.35	102.16	103.97	118.41	174.56
25	133.80	136.06	138.31	156.38	221.59
30	160.60	163.48	166.37	189.45	276.25
35	226.28	230.33	234.39	266.84	391.93
40	267.95	272.71	277.46	315.53	456.41
45	307.10	312.72	318.33	363.24	540.67
50	351.27	357.77	364.26	416.23	624.21
60	437.97	446.53	455.10	523.65	803.15
70	656.21	668.43	680.65	778.43	1,160.60
80	790.99	805.63	820.26	937.31	1,405.84
90	942.58	959.88	977.18	1,115.58	1,697.99
100	1,061.86	1,082.68	1,103.50	1,270.05	1,969.88

TABLE IX: Experiment 2 - Percentage performance speedup in CPU cycles of APQC-100, APQC-50, and APQC-10 vs. PQC for the average epoch in-session

Group Size	H-100 vs PQC	H-50 vs PQC	H-10 vs PQC
2	24.66%	23.40%	13.30%
3	30.00%	28.88%	19.94%
4	31.56%	30.44%	21.47%
5	31.19%	30.12%	21.56%
10	38.26%	37.18%	28.53%
15	43.72%	42.74%	34.84%
20	41.48%	40.44%	32.16%
25	38.60%	37.58%	29.43%
30	40.82%	39.78%	31.42%
35	41.23%	40.20%	31.92%
40	40.25%	39.21%	30.87%
45	42.16%	41.12%	32.82%
50	42.68%	41.64%	33.32%
60	44.40%	43.34%	34.80%
70	42.41%	41.35%	32.93%
80	42.69%	41.65%	33.33%
90	43.47%	42.45%	34.30%
100	45.04%	43.98%	35.53%

VI. Conclusions and Recommendations

This work showed that the APQC-MLS Combiner can achieve a scalable trade off between the performance of traditional MLS and the PQC security offered by using a PQC cipher suite on MLS. The data supports that for groups

between 2 and 100 members (the tested parameters), the computational work required to start and maintain an MLS group under the APQC-MLS Combiner protocol is less than that of starting and maintaining an MLS group using a post-quantum cipher suite with equivalent FS epochs, largely by a margin of 30-50 percent. It also supports the advantages of APQC-MLS in the contexts of CPU cycle consumption per epoch by margins of primarily 30-45 percent, and control message size requirements by margins of 68-80 percent. Implementers can expect advantages across all group sizes, with the flexibility of adjusting the **Full Update** frequency to meet PQC security needs without sacrificing computational and data transmission efficiency. These conclusions support the viability of achieving efficient post-quantum secure communications.

References

- [1] *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, Federal Information Processing Standards Publication, National Institute of Standards and Technology, 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>
- [2] *Module-Lattice-Based Digital Signature Standard*, Federal Information Processing Standards Publication, National Institute of Standards and Technology, 2024.
- [3] Joël, B. Hale, M. Mularczyk, and X. Tian, "Flexible Hybrid PQ MLS Combiner," Internet Engineering Task Force, Internet-Draft, Feb. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-mls-combiner/00/>
- [4] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8446>
- [5] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, "TLS Encrypted Client Hello," Internet Engineering Task Force, Internet-Draft draft-ietf-tls-esni-25, Jun. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-tls-esni/25/>
- [6] K. Seo and S. Kent, "Security Architecture for the Internet Protocol," RFC 4301, Dec. 2005. [Online]. Available: <https://www.rfc-editor.org/info/rfc4301>
- [7] T. Wiggers, S. Celi, P. Schwabe, D. Stebila, and N. Sullivan, "KEM-based Authentication for TLS 1.3," Internet Engineering Task Force, Internet-Draft, Oct. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/04/>
- [8] D. Connolly, "ML-KEM Post-Quantum Key Agreement for TLS 1.3," Internet Engineering Task Force, Internet-Draft, Nov. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-connolly-tls-mlkem-key-agreement/05/>
- [9] B. Westerbaan and D. Stebila, "X25519Kyber768Draft00 Hybrid Post-Quantum Key Agreement," Internet Engineering Task Force, Internet-Draft draft-tls-westerbaan-xyber768d00-03, Sep. 2023, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-tls-westerbaan-xyber768d00/03/>
- [10] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, "Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3," Internet Engineering Task Force, Internet-Draft draft-kwiatkowski-tls-ecdhe-mlkem-03, Dec. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-kwiatkowski-tls-ecdhe-mlkem/03/>
- [11] P. Kampanakis and G. Ravago, "Post-quantum Hybrid Key Exchange with ML-KEM in the Internet Key Exchange Protocol Version 2 (IKEv2)," Internet Engineering Task Force, Internet-Draft, Nov. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-kampanakis-ml-kem-ikev2/09/>
- [12] G. Wang, "Post-quantum Hybrid Key Exchange in the IKEv2 with FrodoKEM," Internet Engineering Task Force, Internet-

- Draft, Oct. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-wang-hybrid-kem-ikev2-frodo/02/>
- [13] P. Kampanakis, D. Stebila, and T. Hansen, "PQ/T Hybrid Key Exchange in SSH," Internet Engineering Task Force, Internet-Draft, Jan. 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-sshm-mlkem-hybrid-kex/00/>
- [14] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH," in *Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies*, ser. CoNEXT '20, 2020, p. 149–156. [Online]. Available: <https://doi.org/10.1145/3386367.3431305>
- [15] B. Dowling and B. Hale, "Authenticated continuous key agreement: Active MitM detection and prevention," Cryptology ePrint Archive, Paper 2023/228, 2023. [Online]. Available: <https://eprint.iacr.org/2023/228>
- [16] K. Cohn-Gordon, C. Cremers, and L. Garratt, "Post-compromise security," Cryptology ePrint Archive, Paper 2016/221, 2016. [Online]. Available: <https://eprint.iacr.org/2016/221>
- [17] F. Günther, B. Hale, T. Jager, and S. Lauer, "0-rtt key exchange with full forward secrecy," *IACR Cryptol. ePrint Arch.*, vol. 2017, p. 223, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:19973797>
- [18] C. Cremers, B. Hale, and K. Kohbrok, "The complexities of healing in secure group messaging: Why Cross-Group effects matter," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 1847–1864. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/cremers>
- [19] J. Alwen, S. Coretti, D. Jost, and M. Mularczyk, "Continuous Group Key Agreement with Active Security," in *Theory of Cryptography: 18th International Conference, TCC 2020*. Springer-Verlag, 2020, p. 261–290.
- [20] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the signal messaging protocol," in *2017 IEEE European Symposium on Security and Privacy*, 2017, pp. 451–466.
- [21] K. Cohn-Gordon, C. Cremers, and L. Garratt, "On post-compromise security," in *2016 IEEE 29th Computer Security Foundations Symposium (CSF)*, 2016, pp. 164–178.
- [22] K. Milner, C. Cremers, J. Yu, and M. Ryan, "Automatically detecting the misuse of secrets: Foundations, design principles, and applications," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, 2017, pp. 203–216.
- [23] B. Dowling and B. Hale, "There can be no compromise: The necessity of ratcheted authentication in secure messaging," Cryptology ePrint Archive, Paper 2020/541, 2020. [Online]. Available: <https://eprint.iacr.org/2020/541>
- [24] E. M. Fondevik, B. Hale, and X. Tian, "Guardianship in group key exchange for limited environments," Cryptology ePrint Archive, Paper 2023/1761, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1761>
- [25] K. Cohn-Gordon, C. Cremers, L. Garratt, J. Millican, and K. Milner, "On ends-to-ends encryption: Asynchronous group messaging with strong security guarantees," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18, 2018, p. 1802–1819. [Online]. Available: <https://doi.org/10.1145/3243734.3243747>
- [26] K. Hashimoto, S. Katsumata, and T. Wiggers, "Bundled authenticated key exchange: A concrete treatment of (post-quantum) signal's handshake protocol," Cryptology ePrint Archive, Paper 2025/040, 2025. [Online]. Available: <https://eprint.iacr.org/2025/040>
- [27] F. Linker, R. Sasse, and D. Basin, "A formal analysis of apple's iMessage PQ3 protocol," Cryptology ePrint Archive, Paper 2024/1395, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1395>
- [28] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, "The Messaging Layer Security (MLS) Protocol," RFC 9420, Jul. 2023. [Online]. Available: <https://www.rfc-editor.org/info/rfc9420>
- [29] K. Bhargavan, R. Barnes, and E. Rescorla, "TreeKEM: Asynchronous Decentralized Key Management for Large Dynamic Groups A protocol proposal for Messaging Layer Security (MLS)," Inria Paris, Research Report, May 2018, provides a technical description of the TreeKEM. [Online]. Available: <https://inria.hal.science/hal-02425247>
- [30] N. Alnahawi, J. Alperin-Sheriff, D. Apon, and A. Wiesmaier, "NICE-PAKE: On the security of KEM-based PAKE constructions without ideal ciphers," Cryptology ePrint Archive, Paper 2024/1957, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1957>
- [31] R. Barnes, K. Bhargavan, B. Lipp, and C. A. Wood, "Hybrid Public Key Encryption," RFC 9180, Feb. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9180>
- [32] M. Ounsworth, A. Wussler, and S. Kousidis, "Combiner function for hybrid key encapsulation mechanisms (Hybrid KEMs)," Internet Engineering Task Force, Internet-Draft, Jan. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ounsworth-cfrg-kem-combiners/05/>
- [33] D. Connolly, P. Schwabe, and B. Westerbaan, "X-Wing: general-purpose hybrid post-quantum KEM," Internet Engineering Task Force, Internet-Draft draft-connolly-cfrg-xwing-kem-06, Oct. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/06/>
- [34] B. Westerbaan and C. A. Wood, "X25519Kyber768Draft00 hybrid post-quantum KEM for HPKE," Internet Engineering Task Force, Internet-Draft, May 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-westerbaan-cfrg-hpke-xyber768d00/03/>
- [35] R. Mahy, "Messaging Layer Security Ciphersuite using XWing Key Exchange Mechanism," Internet Engineering Task Force, Internet-Draft, Mar. 2024, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-mahy-mls-xwing/00/>
- [36] openmls. (2020) openmls. [Online]. Available: <https://github.com/openmls/openmls>
- [37] S. Lenz, "Evaluation of the messaging layer security protocol: A performance and usability study," Master's thesis, Linköping University, Department of Electrical Engineering, Information Coding, 2020.
- [38] B. Heisler. (2017) criterion.rs. [Online]. Available: <https://github.com/bheisler/criterion.rs>